

GPT-5.6 COURSE CHEATSHEET

O'Reilly Live Training — Getting Started with GPT-5.6

Instructor: Lucas | Responses API · Parameters · Tools · Prompting · Agents | 2026

1. MODEL VARIANTS (SOL / TERRA / LUNA, RELEASED 2026-07-09)

gpt-5.6-sol

FLAGSHIP

- \$5.00 / \$30.00 per 1M tokens (in/out)
- Frontier coding, agentic & professional reasoning
- Absorbs the old dedicated-coding role (gpt-5.3-codex)
- Alias: bare `gpt-5.6` routes here

gpt-5.6-terra

- \$2.50 / \$15.00 per 1M tokens (in/out)
- ~GPT-5.5-class quality at ~half the price
- General business/agentic tasks
- Replace: o4-mini

gpt-5.6-luna

- \$1.00 / \$6.00 per 1M tokens (in/out)
- Cheapest, fastest tier
- High-volume chat, extraction, classify
- Replace: gpt-4.1-nano

⚠ Context / Pricing Notes (all three tiers)

Context window **1,050,000** tokens, max output **128,000** tokens. Prompts over 272K input tokens incur a long-context surcharge (~2x input / ~1.5x output rate). System-card names differ from API identifiers — check developers.openai.com/api/docs/guides/latest-model for the official mapping table.

2. RESPONSES API (RECOMMENDED)

Basic Call

```
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-5.6-sol",
    input="Your prompt here",
    reasoning={"effort": "medium"},
    text={"verbosity": "medium"}
)
print(response.output_text)
```

```
first = client.responses.create(
    model="gpt-5.6-sol",
    conversation=conversation.id,
    input="Design an LRU cache in
Python",
    reasoning={"effort": "high"}
)

# Follow-up – shares context via
conversation id
follow_up = client.responses.create(
    model="gpt-5.6-sol",
    conversation=conversation.id,
    input="Add thread-safety."
)

# Legacy chaining (still works):
previous_response_id=first.id
```

Why Responses API over Chat Completions?

- **CoT persistence** via `conversation= id` (or `legacy previous_response_id`)
- Fewer reasoning tokens re-generated
- Higher cache hit rates
- Lower latency on follow-ups
- Conversation-attached responses persist indefinitely (no 30-day TTL)

Unsupported in GPT-5.6

- `temperature` — not supported
- `top_p` — not supported
- `logprobs` — not supported

Use instead:

- `reasoning: {effort: ...}`
- `text: {verbosity: ...}`
- `max_output_tokens`

3. REASONING EFFORT PARAMETER

`reasoning={"effort": "..."} — Default: "medium" (all 3 tiers: Sol/Terra/Luna)`

Value	Speed	Reasoning Tokens	Best For
"none"	Fastest TTFT	None	Latency-critical: voice turns, fast retrieval, simple classification
"low"	Fast	Light	Quick responses, GPT-4.1 migrations
"medium"	Balanced	Moderate	Default — general tasks
"high"	Slow	Most	Complex math, multi-step planning, difficult code
"xhigh"	Slower	Very high	Hardest async/agentic tasks (top tier added in GPT-5.5)
"max"	Slowest	Highest	Hardest problems — often paired with <code>reasoning.mode:"pro"</code>

`reasoning.mode: "pro"` (replaces the old `gpt-5.5-pro` model)

GPT-5.6 drops the separate `-pro` model. Instead, set `reasoning: {"effort": "high", "mode": "pro"}` on `gpt-5.6-sol` — `mode` and `effort` are independent axes, and `pro-mode` tokens bill at Sol's standard per-token rates (no separate pricing row). Documented across all three tiers, but foregrounded for Sol. Reserve for one hardest-reasoning demo, not a default.

Note: `"minimal"` is no longer a supported value on any GPT-5.6 tier — use `"none"` or `"low"` instead.

4. VERBOSITY PARAMETER

`text={"verbosity": "..."} — Default: "medium"`

Value	Output Length	Best For
"low"	Lowest of the three	Terse UX, SQL queries, concise code, minimal prose
"medium"	Baseline	Balanced explanations (default)
"high"	Highest of the three	Audits, teaching, hand-offs, detailed documentation

Measure, Don't Assume — and Mixed Verbosity Strategy

Exact token counts per verbosity level vary by prompt and model generation — benchmark live against your own prompts rather than relying on fixed numbers. The trend (low < medium < high) is stable across the GPT-5.x line. Cursor-style pattern: set global `verbosity="low"` for prose, but instruct "use high verbosity for code tools" in the system prompt to get concise explanations + detailed, well-commented code.

5. NEW TOOL FEATURES

Freeform / Custom Tool Calling (NEW)

```
# Raw text payloads – no JSON wrapping
response = client.responses.create(
    model="gpt-5.6-sol",
    input="Use code_exec to find
factorial of 10.",
    tools=[{
        "type": "custom",
        "name": "code_exec",
        "description": "Executes
arbitrary Python code"
    }]
)
# Access: response.output[1].input
(raw code string)
# Feed back: {"type":
"function_call_output",
#           "call_id":
tool_call.call_id,
#           "output":
"result_string"}
```

Context-Free Grammar (CFG) Constraints (NEW)

```
# Constrain output to exact syntax
(Lark or Regex)
tools=[{
    "type": "custom",
    "name": "sql_tool",
    "description": "SQL tool – reason
to conform",
    "format": {
        "type": "grammar",
        "syntax": "lark", # or "regex"
        "definition": r"""
start: "SELECT" SP cols SP "FROM"
      SP table SP "WHERE" SP filter
cols: IDENTIFIER ("," SP IDENTIFIER)*
IDENTIFIER: /[A-Za-z_][A-Za-z0-9_]*/
SP: " "
      """
    }
}]
```

Freeform Tool — Key Points

- Use for Python, SQL, shell, config — anything where JSON wrapping is unnatural
- Model emits `ResponseCustomToolCall` with `.input` = raw string
- ⚠ Does **NOT** support parallel tool calling
- Great for: code sandboxes, SQL databases, shell envs
- Feed result back via `function_call_output` to continue

CFG — Key Points & Best Practices

- Lark: terminals (UPPER) matched first; rules (lower) combine them
- Regex: provide pattern string directly
 - One bounded terminal for free-text between anchors
 - Don't split free-text across multiple terminals
- Use `SP = " "` explicitly; avoid `.*` wildcards
- If drifting OOD → tighten grammar + bump reasoning effort

Allowed Tools — Selective Access (NEW)

- Define full toolkit, restrict active subset per request
- Use for safety, predictability, preventing unintended calls

```
tool_choice={"type":"allowed_tools","mode":"auto","tools":
[{"type":"function","name":"safe_tool"}]}
```

Tool Preambles

- GPT-5 trained to give upfront plans + progress updates
- Trigger: "Before calling a tool, explain why"
- System prompt: "Rephrase goal, outline plan, narrate each step, summarize at end"
- Useful for debugging and transparency

6. MIGRATION GUIDE

Migrating From Previous Models

From Model	Migrate To	Recommended Settings
o3	gpt-5.6-sol	reasoning.effort: "medium" or "high"
gpt-4.1	gpt-5.6-sol	reasoning.effort: "low" ("minimal" was retired — map to "none" or "low")
o4-mini	gpt-5.6-terra	Default settings + prompt tuning
gpt-4.1-nano	gpt-5.6-luna	Default settings + prompt tuning

7. PROMPTING BEST PRACTICES

Controlling Agentic Eagerness

Less eager (quick, focused):

- Start broad, fan out to focused subqueries in parallel
- Stop as soon as you can name exact content to change
- Early stop: top hits converge ~70% on one area
- Batch search → minimal plan → complete task

More eager (autonomous, persistent):

- "Keep going until query is completely resolved"
- "Never stop when encountering uncertainty"
- "Decide, proceed, document assumptions for user"

Instruction Following Rules

- Keep instructions **non-contradictory**
- Priority order: Security (CRITICAL) → Logic (HIGH) → Performance (MEDIUM) → Style (LOW)
- Specify output format explicitly
- Use structured system prompts with clear sections
- Markdown not used by default — must prompt for it
- Inline math: `\( \)`, block math: `\[ \]`

Self-Reflection for High Quality

Add to system prompt:

- Create a quality rubric with 5-7 categories internally
- Evaluate and iterate until hitting top marks in ALL categories
- Don't show rubric to user — it's for internal reasoning
- Great for: one-shot app generation, frontend code, complex outputs

Metaprompting

Ask GPT-5 to improve its own prompts:

- *"Here's a prompt: {prompt}. Desired behavior: {X}. It instead {Y}. What minimal edits encourage improvement?"*
- Use `reasoning.effort: "high"` for metaprompting tasks
- Keep as much of the original prompt intact as possible
- Ask for specific phrases to add or remove

8. AGENTIC CAPABILITIES

Web Search Tool

```
response = client.responses.create(
    model="gpt-5.6-sol",
    tools=[{"type": "web_search"}],
    input="Latest AI tool releases this
quarter?"
)
print(response.output_text)

# Chain: Markdown table from results
(cost-optimized follow-up)
table_response =
client.responses.create(
    model="gpt-5.6-terra",
    previous_response_id=response.id,
    input=[{"role": "user",
            "content": "Put results in
a markdown table."}]
)
```

Multi-turn Agentic Loop

```
def run_conversation(messages,
prev_id=None):
    response = client.responses.create(
        model="gpt-5.6-sol",
        input=messages,
        tools=TOOLS,
        previous_response_id=prev_id
    )
    tool_call = response.output[1] \
        if len(response.output) > 1
    else None

    if tool_call and
tool_call.type=="custom_tool_call":
        result =
execute_tool(tool_call.input)
        messages.append({
            "type":
"function_call_output",
            "call_id":
tool_call.call_id,
            "output": result
        })
        return
run_conversation(messages, response.id)
return response # base case: no
more tool calls
```

Retail Agent Pattern

1. Authenticate user (email or name+zip)
2. Retrieve relevant data
3. Confirm before consequential actions
4. Execute with reasoning: "none"
5. One tool call at a time

SWE-Bench Pattern

- Use reasoning: "high"
- Verify changes extremely thoroughly
- Account for hidden test edge cases
- Make as many tool calls as needed
- Triple-check before ending turn

Frontend Dev Pattern

- Stack: Next.js + TypeScript + Tailwind + shadcn/ui
- Icons: Lucide, Animation: Motion
- 4px spacing multiples, max 4-5 font sizes
- Skeletons for loading states
- Self-reflection rubric for quality

9. QUICK REFERENCE & GOTCHAS

DO THIS

- Use **Responses API** (`client.responses.create()`), not Chat Completions
- Use `client.conversations.create()` for multi-turn (legacy `previous_response_id` still works)
- Set `reasoning.effort: "none"` explicitly for speed-critical paths
- Use `text.verbosity: "high"` for thorough code docs/audits
- Keep CFG grammars **simple** — complex ones get API-rejected
- Set `parallel_tool_calls=False` with custom tools
- Use `gpt-5.6-terra` or `gpt-5.6-luna` for cost-sensitive follow-up processing of Sol outputs

AVOID THIS

- Using `temperature`, `top_p`, `logprobs` — not supported in GPT-5.6
- Using `reasoning.effort: "minimal"` — retired; not supported on any GPT-5.6 tier
- Parallel tool calling with custom/freeform tools — not supported
- Splitting free-text across multiple CFG terminals — use one bounded terminal
- Contradictory instructions in the same system prompt
- Complex CFG grammars without testing at high reasoning effort first
- Re-generating full CoT on every turn — use `conversation=` or `previous_response_id`
- Using Chat Completions for GPT-5.6 — missing CoT persistence

Key Resources

System Card: openai.com/index/gpt-5-system-card/ | Prompting Guide: cookbook.openai.com/examples/gpt-5/gpt-5_prompting_guide | API Docs: developers.openai.com/api/docs/guides/latest-model | Model Line: developers.openai.com/api/docs/models/gpt-5.6-sol | Lark IDE: www.lark-parser.org/ide/